

Universidad de San Carlos de Guatemala

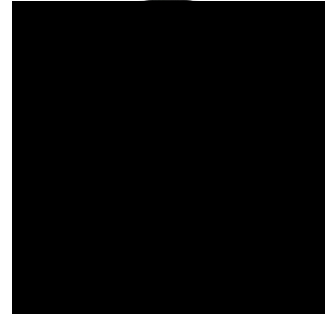
Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Organización de Lenguajes y Compiladores 2

Sección: A

Docente: Mario Moisés Ramírez Tobar



“Manual Técnico Práctica 1”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 24 de agosto de

2026

Manual de Tecnico Codex Latinus

1. Introduccion

El presente documento es el Manual Tecnico del Compilador Codex Latinus, desarrollado como Practica 1 del curso Organizacion de Lenguajes y Compiladores 2. El compilador esta implementado en Java 21, utilizando ANTLR 4.13.2 para la generacion del analizador lexico y sintactico, con una interfaz grafica construida en Java Swing y reportes graficos generados con Graphviz.

Codex Latinus es un lenguaje de programacion de sintaxis inspirada en el latin, disenado para ejercitar las fases classicas de un compilador: analisis lexico, analisis sintactico, analisis semantico y recorrido del Arbol de Sintaxis Abstracta. El lenguaje incorpora cinco tipos primitivos con jerarquia de conversion implicita, arreglos, estructuras anidables, funciones con y sin retorno, tres tipos de ciclos, condicionales encadenados y funciones especiales de entrada y salida.

El entregable incluye ademas tres capacidades exigidas por el enunciado: la graficacion del Arbol de Sintaxis Abstracta y de la tabla de simbolos, la simulacion paso a paso de la pila de analisis con navegacion hacia adelante y hacia atras, y la traduccion del codigo fuente al idioma PigLatin recorriendo el arbol.

2. Arquitectura del Sistema

2.1 Estructura de paquetes

El proyecto sigue una arquitectura en capas, organizada bajo Maven. Cada paquete corresponde a una fase o responsabilidad concreta del compilador:

```
com.mycompany.codexlatinus.gramatica
    CodexLatinus.g4      -> Gramatica fuente (src/main/antlr4)
    CodexLatinusLexer    -> Generado por ANTLR
    CodexLatinusParser   -> Generado por ANTLR
    CodexLatinusBaseListener -> Generado por ANTLR

com.mycompany.practica_1_compi_2_mynor_monzon_1s_2026

    tipos/
        TipoPrimitivo.java -> Enum con la jerarquia de tipos
        Tipo.java          -> Tipo completo: primitivo, arreglo o
    estructura
        Operador.java      -> Operadores agrupados por categoria
        TablaTipos.java    -> Reglas de compatibilidad e inferencia

    ast/
        Nodo.java          -> Clase base abstracta del AST
        NodoPrograma.java  -> Raiz del arbol
        expresion/
            instruccion/    -> 21 nodos de instruccion

    analizador/
        ConstructorAST.java -> Listener que construye el AST propio
        Compilador.java     -> Orquestador de todas las fases
```

```

errores/
    TipoError.java          -> Enum: LEXICO, SINTACTICO, SEMANTICO
    ErrorCompilacion.java -> Modelo de error con linea y columna
    RecolectorErrores.java -> Lista central compartida
    EscuchaErrores.java    -> Captura los errores de ANTLR
    ModoPanico.java        -> Recuperacion ante errores sintacticos

simbolos/
    RolSimbolo.java         -> Clasificacion del simbolo
    Simbolo.java            -> Entrada de la tabla
    Ambito.java             -> Nivel de alcance con cadena al padre
    TablaSimbolos.java      -> Pila de ambitos e historial plano

semantico/
    AnalizadorSemantico.java -> Validaciones en cinco pasadas

pila/
    OperacionPila.java      -> Enum: SHIFT, REPLACE, ACCEPT
    PasoPila.java           -> Fotografia de la pila en un paso
    RegistradorPila.java    -> Simula la pila con eventos del Listener

traductor/
    ReglasPigLatin.java     -> Leyes de consonantes, vocales y porcina
    TraductorPigLatin.java -> Recorre el AST y regenera el codigo

reportes/
    GeneradorDot.java       ->Codigo DOT del AST, tabla y pila

interfaz/
    VentanaPrincipal.java -> Ventana principal Swing
    ResaltadorSintaxis.java -> Coloreado con el lexer de ANTLR
    GestorArchivos.java    -> Abrir, guardar y descargar
    VisorDot.java          -> Muestra los graficos generados
    VentanaPila.java       -> Navegacion paso a paso de la pila

```

2.2 Flujo de compilacion

El procesamiento de un archivo con extension .lat sigue las etapas que se describen a continuacion. La clase Compilador orquesta todas ellas y corta el proceso cuando continuar no tendria sentido:

Etapas	Componente	Descripcion
1	CodexLatinusLexer	Tokenizacion del codigo fuente. Reconoce lexemas y genera el flujo de tokens con su linea y columna exactas. Los errores lexicos se recolectan antes de parsear.
2	CodexLatinusParser	Analisis sintactico LL(*) mediante ANTLR4. Valida la estructura gramatical y produce el parse tree. Ante un error, ModoPanico sincroniza y permite continuar.
3	ConstructorAST	Listener que recorre el parse tree en post orden y construye el AST propio del compilador, mucho mas compacto que el

Etapas	Componente	Descripcion
		arbol de ANTLR.
4	RegistradorPila	Segundo recorrido del parse tree que simula las operaciones de una pila de analisis y guarda una fotografia por cada paso.
5	AnalizadorSemantico	Recorre el AST en cinco pasadas. Construye la tabla de simbolos, resuelve tipos y aplica todas las validaciones del enunciado.
6	GeneradorDot	Produce el codigo DOT del AST, de la tabla de simbolos y de la pila para que Graphviz genere las imagenes.
7	TraductorPigLatin	Recorre el AST ya validado y regenera el codigo aplicando las leyes de traduccion.
8	VentanaPrincipal	Presenta el editor con coloreado, la consola, los reportes graficos y la traduccion.

Es importante notar que las fases lexica y sintactica detienen el proceso si encuentran errores. La razon es que un AST construido sobre un arbol de analisis incompleto produciria errores semanticos falsos que confundirian al usuario. En cambio, el analisis semantico nunca se detiene: recorre el arbol completo acumulando todos los errores que encuentra.

3. Analizador Lexico

3.1 Descripcion general

El analizador lexico se genera automaticamente a partir del archivo CodexLatinus.g4 mediante el plugin antlr4-maven-plugin. La seccion del lexer de la gramatica define las palabras reservadas, los operadores, los signos de agrupacion y los literales del lenguaje.

Codex Latinus es un lenguaje sensible a mayusculas y minusculas. Esta caracteristica tiene una consecuencia directa en el diseno: la palabra finis en minusculas cierra bloques, mientras que FINIS en mayusculas cierra el programa completo. Ambas se definen como tokens distintos precisamente por esa razon.

Las palabras reservadas se declaran antes que la regla de identificadores porque ANTLR resuelve los empates de igual longitud a favor de la regla declarada primero. Si el orden fuera el inverso, toda palabra reservada se reconoceria como un identificador cualquiera.

3.2 Definiciones de expresiones regulares

Token	Expresion Regular	Descripcion
DECIMAL	<code>[0-9]+ '.' [0-9]+</code>	Literal decimal de punto flotante
ENTERO	<code>[0-9]+</code>	Literal entero
CADENA	<code>'"' ('\\ ' . ~["\\r\n]) * ''</code>	Cadena entre comillas dobles, con soporte de secuencias de escape
CHARACTER	<code>'\'' ('\\ ' . ~['\\r\n]) '\'</code>	Caracter unico entre comillas simples
ID	<code>[a-zA-Z_] [a-zA-Z_0-9] *</code>	Identificador valido
COMENTARIO_BLOQUE	<code>'###' .*? '###'</code>	Comentario de varias lineas, se descarta
COMENTARIO_LINEA	<code>'// ' ~[\r\n] *</code>	Comentario de una linea, se descarta
ESPACIOS	<code>[\t\r\n] +</code>	Espacios en blanco, se descartan

La regla DECIMAL se declara antes que ENTERO porque ANTLR aplica la coincidencia mas larga. Sin ese orden, el literal 9.81 se dividiria en tres tokens: el entero 9, el punto y el entero 81.

3.3 Tokens reconocidos

Marcadores de seccion:

Lexema	Token	Descripcion
VARIABLES	VARIABLES	Abre la seccion global de declaraciones y tambien el bloque local dentro de una funcion
MUNERA	MUNERA	Abre la seccion de definicion de funciones
MAIOR	MAIOR	Abre la seccion principal, obligatoria
FINIS	FIN_PROGRAMA	Cierra el programa completo, en mayusculas

Palabras reservadas de declaracion y control:

Lexema	Token	Descripcion
esto	ESTO	Declaracion de una variable simple
series	SERIES	Declaracion de un arreglo
estructura	STRUCTURA	Definicion de una estructura

Lexema	Token	Descripcion
finis	FINIS	Cierre de bloque, en minusculas
si	SI	Inicio de una sentencia condicional
aliter	ALITER	Rama alternativa del condicional, con o sin condicion
dum	DUM	Ciclo mientras, condicion al inicio
facere	FACERE	Ciclo hacer mientras, condicion al final
per	PER	Ciclo para, con inicializacion, condicion y actualizacion
perge	PERGE	Continua con la siguiente vuelta del ciclo
interrumpe	INTERRUMPE	Corta la ejecucion del ciclo
actio	ACTIO	Definicion de funcion sin valor de retorno
ratio	RATIO	Definicion de funcion con valor de retorno
reddere	REDDERE	Retorno de una funcion
non	NON	Negacion logica

Tipos de datos:

Lexema	Token	Tipo en Codex Latinus
numerus	NUMERUS	Numero entero
decimalis	DECIMALIS	Numero de punto flotante
textum	TEXTUM	Cadena de caracteres
littera	LITTERA	Caracter unico
bool	BOOL	Valor booleano
verum	VERUM	Literal booleano verdadero
falsus	FALSUS	Literal booleano falso

Operadores:

Lexema	Token	Descripcion
+	MAS	Suma o concatenacion de cadenas
-	MENOS	Resta o negacion unaria
*	POR	Multiplicacion
/	DIV	Division

Lexema	Token	Descripcion
++	MASMAS	Incremento en uno
--	MENOSMENOS	Decremento en uno
=	IGUAL	Asignacion
==	IGUALIGUAL	Comparacion de igualdad
!=	DIFERENTE	Comparacion de desigualdad
<	MENOR	Menor que
>	MAYOR	Mayor que
<=	MENORIGUAL	Menor o igual que
>=	MAYORIGUAL	Mayor o igual que
&&	AND	Conjuncion logica
	OR	Disyuncion logica
>>	MAYORMAYOR	Funcion especial de impresion
<<	MENORMENOR	Funcion especial de lectura

Los operadores de dos caracteres se declaran antes que los de uno solo. De lo contrario, la secuencia >> se reconoceria como dos tokens MAYOR consecutivos en lugar del operador de impresion.

Signos de agrupacion y puntuacion:

Lexema	Token	Descripcion
()	PAR_A / PAR_C	Agrupacion de expresiones y lista de parametros
{ }	LLAVE_A / LLAVE_C	Cuerpo de bloques, estructuras y listas de valores
[]	COR_A / COR_C	Dimension e indice de arreglos, bloque VARIABLES local
;	PYC	Fin de instruccion o separador de campos
,	COMA	Separador de elementos y de campos
.	PUNTO	Acceso a atributo de estructura
:	DOS_PUNTOS	Separa el nombre del tipo en una declaracion

3.4 Manejo de errores lexicos

Cuando el lexer encuentra un caracter que no pertenece a ninguna de las reglas definidas, ANTLR emite un error de reconocimiento de token. La clase EscuchaErrores intercepta ese evento, lo clasifica como error lexico y lo registra en el RecolectorErrores con la linea y la columna exactas.

Para que todos los errores lexicos se reporten en una sola pasada, la clase Compilador invoca el metodo fill del flujo de tokens antes de iniciar el analisis sintactico. Sin esa llamada el parser se detendria en el primer token invalido y los caracteres problematicos posteriores nunca se examinarian.

4. Gramatica del Lenguaje Codex Latinus

4.1 Descripcion general

La gramatica esta definida en el archivo CodexLatinus.g4 y es procesada por ANTLR 4.13.2 para generar un analizador sintactico descendente de tipo LL(*). La gramatica es no ambigua gracias al uso de delimitadores obligatorios en todos los bloques y al ordenamiento de las alternativas de la regla de expresiones, que codifica la precedencia de los operadores.

Se utiliza la siguiente convencion en la notacion:

- Los no terminales se escriben en minusculas con notacion camel (por ejemplo declaracionVariable).
- Los terminales se escriben en mayusculas (por ejemplo ESTO, LLAVE_A).
- El simbolo dos puntos separa la cabeza de la produccion de su cuerpo.
- El simbolo | separa alternativas de una misma produccion.
- El simbolo ? indica que el elemento es opcional, y * que puede repetirse cero o mas veces.
- Las etiquetas precedidas de # nombran cada alternativa para que ANTLR genere un metodo distinto en el Listener.

4.2 Estructura general del programa

Un programa valido se compone de tres secciones. Las dos primeras son opcionales y la ultima es obligatoria:

```
programa
    : seccionVariables? seccionMunera? seccionMaior EOF
    ;

seccionVariables
    : VARIABLES MAYOR declaracion*
    ;

seccionMunera
    : MUNERA MAYOR definicionFuncion*
    ;

seccionMaior
```



```

: MAIOR MAYOR instruccion* FIN_PROGRAMA PYC?
;

```

Los marcadores de seccion se escriben con el signo mayor que a continuacion del nombre, como en VARIABLES mayor que. Se tokenizan como dos elementos separados en lugar de un unico token, lo que permite tolerar un espacio entre ambos sin alterar el significado.

4.3 Producciones de la gramatica

Tipos y declaraciones:

```

tipo
: NUMERUS | DECIMALIS | TEXTUM | LITTERA | BOOL | ID
;

declaracion
: declaracionVariable
| declaracionArreglo
| definicionEstructura
;

declaracionVariable
: ESTO ID DOS_PUNTOS ID literalEstructura PYC?      # DeclEstructura
| ESTO ID DOS_PUNTOS valorBooleano expresion? PYC?  # DeclBooleana
| ESTO ID DOS_PUNTOS tipo expresion PYC?            # DeclConValor
| ESTO ID DOS_PUNTOS tipo PYC?                      # DeclSinValor
;

declaracionArreglo
: SERIES ID dimension? DOS_PUNTOS tipo listaValores? PYC?
                                                    # ArregloTipado
| SERIES ID dimension? DOS_PUNTOS listaValores PYC? # ArregloInferido
;

dimension
: COR_A expresion COR_C
;

listaValores
: LLAVE_A (valorLista (COMA valorLista)*)? COMA? LLAVE_C
;

valorLista
: literalEstructura
| expresion
;

```

El orden de las alternativas de declaracionVariable no es arbitrario. La alternativa de estructura va primero porque se distingue por la llave de apertura que la sigue; la booleana va antes que las generales porque los literales verum y falsus ocupan la posicion del tipo en la forma abreviada del lenguaje.

Estructuras:

```
definicionEstructura
: STRUCTURA ID LLAVE_A campoEstructura* LLAVE_C FINIS PYC?
;

campoEstructura
: ESTO ID DOS_PUNTOS tipo separadorCampo?          # CampoSimple
| ESTO ID DOS_PUNTOS valorBooleano separadorCampo?  #
CampoBooleano
| SERIES ID dimension? DOS_PUNTOS tipo separadorCampo? #
CampoArreglo
;

separadorCampo
: PYC | COMA
;

literalEstructura
: LLAVE_A (asignacionAtributo
  (separadorCampo asignacionAtributo)*)? separadorCampo? LLAVE_C
;

asignacionAtributo
: ID DOS_PUNTOS valorAtributo
;

valorAtributo
: literalEstructura
| listaValores
| dimensionPrimitiva
| expresion
;

dimensionPrimitiva
: (NUMERUS | DECIMALIS | TEXTUM | LITTERA | BOOL)
COR_A expresion COR_C
;
```

Funciones:

```
definicionFuncion
: ACTIO ID PAR_A listaParametros? PAR_C cuerpoFuncion
  FINIS PYC?          # FuncionSinRetorno
| RATIO tipo ID PAR_A listaParametros? PAR_C cuerpoFuncion
  FINIS PYC?          # FuncionConRetorno
;

cuerpoFuncion
: LLAVE_A bloqueVariablesLocal? instruccion* LLAVE_C
;

bloqueVariablesLocal
: VARIABLES COR_A declaracion* COR_C
;
```

```

listaParametros
    : parametro (COMA parametro)*
    ;

parametro
    : ESTO ID DOS_PUNTOS tipo
    ;

llamadaFuncion
    : ID PAR_A listaArgumentos? PAR_C
    ;

```

Instrucciones y control de flujo:

```

instruccion
    : asignacion | incremento | condicional
    | cicloDum | cicloFacere | cicloPer
    | perge | interrompe | reddere
    | imprimir | leer | llamadaInstruccion
    | declaracionVariable | declaracionArreglo
    | definicionEstructura
    ;

bloque
    : LLAVE_A instruccion* LLAVE_C
    ;

objetivo
    : ID sufijoAcceso*
    ;

sufijoAcceso
    : PUNTO ID
    | COR_A expresion COR_C
    ;

condicional
    : SI PAR_A expresion PAR_C bloque
      ramaAliterSi* ramaAliter? FINIS PYC?
    ;

cicloDum
    : DUM PAR_A expresion PAR_C bloque FINIS PYC?
    ;

cicloFacere
    : FACERE bloque DUM PAR_A expresion PAR_C PYC?
    ;

cicloPer
    : PER PAR_A inicializacionPer PYC expresion PYC
      actualizacionPer PAR_C bloque (FINIS PYC)?
    ;

```

```

imprimir
: MAYORMAYOR expresion (MAYORMAYOR? expresion)* PYC?
;

leer
: objetivo MENORMENOR PYC?      # LeerEnVariable
| MENORMENOR PYC?                # LeerDescartado
;

```

La regla instruccion acepta declaraciones de manera deliberada, aunque el enunciado prohíbe declarar variables fuera de las secciones designadas. La razón es la calidad del diagnóstico: si la gramática las rechazara, el usuario recibiría un error sintáctico genérico. Al aceptarlas, el analizador semántico puede emitir el mensaje preciso que indica donde deben ir las declaraciones.

Expresiones:

```

expresion
: PAR_A expresion PAR_C          # ExprAgrupada
| (MENOS | NON) expresion        # ExprUnaria
| expresion (POR | DIV) expresion # ExprMulDiv
| expresion (MAS | MENOS) expresion # ExprSumaResta
| expresion (MENOR | MAYOR | MENORIGUAL
              | MAYORIGUAL) expresion # ExprRelacional
| expresion (IGUALIGUAL | DIFERENTE) expresion # ExprIgualdad
| expresion AND expresion        # ExprAnd
| expresion OR expresion         # ExprOr
| llamadaFuncion                 # ExprLlamada
| objetivo                       # ExprAcceso
| literal                        # ExprLiteral
;

literal
: ENTERO | DECIMAL | CADENA | CARACTER | VERUM | FALSUS
;

```

4.4 Precedencia y asociatividad de operadores

ANTLR 4 resuelve automáticamente la recursividad por la izquierda y asigna la precedencia según el orden en que aparecen las alternativas dentro de una misma regla: la primera alternativa tiene la mayor precedencia. Esto reemplaza el esquema clásico de reglas encadenadas por niveles que se emplea en generadores de analizadores ascendentes.

La precedencia resultante, de mayor a menor, es la siguiente:

Nivel	Operadores	Asociatividad y notas
1	()	Agrupación explícita, máxima precedencia
2	- unario, non	Unarios por la derecha
3	*, /	Izquierda

Nivel	Operadores	Asociatividad y notas
4	+, -	Izquierda. El operador + tambien concatena cadenas
5	<, >, <=, >=	Izquierda. Producen siempre un valor booleano
6	==, !=	Izquierda. Producen siempre un valor booleano
7	&&	Izquierda. Exige booleano en ambos operandos
8		Izquierda. Menor precedencia de todas

La consecuencia practica es que la expresion $a + b * 2$ construye un arbol donde la multiplicacion queda como hijo de la suma, y no al revés. La forma del arbol representa la precedencia sin necesidad de almacenarla explicitamente en los nodos.

4.5 Recuperacion de errores en modo panico

La clase ModoPanico extiende DefaultErrorStrategy de ANTLR y sobrescribe el metodo recover. Cuando el parser encuentra un error del que no puede recuperarse con la estrategia estandar, esta implementacion descarta tokens hasta encontrar uno que marque el cierre de una construccion del lenguaje.

Los tokens de sincronizacion elegidos son los siguientes:

Token	Justificacion
;	Cierra una instruccion. Se consume tambien, porque la instruccion danada ya termino y el analisis debe continuar con la siguiente.
finis	Cierra un bloque de control, una estructura o una funcion.
FINIS	Cierra el programa completo.
}	Cierra el cuerpo de un bloque o de una funcion.

El resultado es que un archivo con varios errores sintacticos reporta todos los que puede detectar en una sola pasada, en lugar de detenerse en el primero. Cabe aclarar que ANTLR suprime los errores consecutivos mientras permanece en modo de recuperacion hasta consumir un token valido, por lo que el numero de errores reportados puede ser menor al numero de errores presentes.

5. Arbol de Sintaxis Abstracta

5.1 Clase base Nodo

El AST del compilador es una estructura propia y no el parse tree que produce ANTLR. La diferencia es sustancial: el parse tree conserva cada token del archivo, incluidos los parentesis, las comas y los punto y coma, mientras que el AST guarda unicamente los elementos que tienen significado para las fases posteriores.

La clase abstracta Nodo define el contrato comun de todos los nodos:

Miembro	Proposito
linea, columna	Posicion en el codigo fuente. Permiten reportar errores semanticos senalando el lugar exacto.
etiqueta()	Metodo abstracto que devuelve el texto que se muestra dentro del nodo al graficar el arbol.
hijos()	Metodo abstracto que devuelve los hijos directos en el orden en que aparecen en el codigo.
aTexto(nivel)	Representacion con sangria para depuracion en consola. Se implementa una sola vez usando los dos metodos anteriores.

La existencia de etiqueta y hijos como metodos abstractos resuelve un problema de mantenimiento: cualquier recorrido generico del arbol, incluida la generacion del codigo DOT para Graphviz, se escribe una sola vez y funciona con nodos que todavia no existen. Agregar un tipo de nodo nuevo no obliga a modificar el generador de reportes.

5.2 Nodos de expresion

Una expresion es todo aquello que produce un valor. La clase abstracta Expresion extiende Nodo y agrega el campo tipoResuelto, que comienza vacio y lo completa el analizador semantico.

Clase	Construccion que representa
Literal	Valor escrito directamente: 10, 9.81, "texto", 'a', verum, falsus
Identificador	Referencia a una variable, parametro o arreglo por su nombre
OperacionBinaria	Operacion con dos operandos, de cualquier categoria
OperacionUnaria	Negacion logica con non, o menos unario
AccesoArreglo	Acceso a un elemento por indice, encadenable
AccesoAtributo	Acceso a un atributo de estructura con punto, encadenable
LlamadaFuncion	Invocacion de una funcion con sus argumentos
LiteralLista	Lista de valores iniciales de un arreglo entre llaves

Clase	Construccion que representa
LiteralEstructura	Instancia de estructura con sus atributos inicializados
AtributoInicializado	Par nombre y valor dentro de una instancia de estructura

El encadenamiento de accesos se representa mediante anidamiento. La expresion persona.carro.placa produce un AccesoAtributo cuyo hijo es otro AccesoAtributo, cuyo hijo a su vez es un Identificador. Este diseno permite cadenas de cualquier profundidad sin casos especiales, tal como confirmo el auxiliar del curso en el foro de dudas.

5.3 Nodos de instruccion

Una instruccion ejecuta una accion pero no produce un valor. Por esa razon la clase abstracta Instruccion no almacena tipo resuelto.

Clase	Construccion que representa
Bloque	Conjunto de instrucciones entre llaves
DeclaracionVariable	Declaracion de una variable simple, con o sin valor inicial
DeclaracionArreglo	Declaracion de un arreglo, con o sin dimension y valores
DefinicionEstructura	Definicion de una estructura con sus campos
CampoEstructura	Un campo dentro de la definicion de una estructura
DefinicionFuncion	Funcion actio o ratio con parametros y cuerpo
Parametro	Parametro formal de una funcion
Asignacion	Asignacion a variable, elemento de arreglo o atributo
IncrementoDecremento	Operadores de incremento y decremento
Condicional	Cadena completa de si, aliter con condicion y aliter final
RamaCondicional	Una rama con condicion dentro de un condicional
CicloDum	Ciclo con la condicion evaluada al inicio
CicloFacere	Ciclo con la condicion evaluada al final
CicloPer	Ciclo con inicializacion, condicion y actualizacion
Perge	Continua con la siguiente vuelta del ciclo
Interrumpe	Corta la ejecucion del ciclo
Reddere	Retorno de funcion, con o sin valor
Imprimir	Impresion de uno o varios valores
Leer	Lectura hacia un destino, o descartando el valor
LlamadaInstruccion	Llamada a funcion usada como instruccion

Dos decisiones de diseño merecen explicación. La primera es que `DefinicionFuncion` almacena las declaraciones locales en una lista separada del cuerpo. Como el enunciado exige que las variables solo se declaren dentro del bloque `VARIABLES` y al inicio de la función, tenerlas en listas distintas convierte esa verificación en una comprobación trivial.

La segunda es que `CicloFacere` invierte el orden de sus campos respecto a `CicloDum`: primero el cuerpo y después la condición. Ese orden refleja la semántica real de la construcción, en la que el cuerpo se ejecuta al menos una vez antes de evaluar la condición.

5.4 Construcción del AST con el patrón Listener

La clase `ConstructorAST` extiende `CodexLatinusBaseListener` y convierte el parse tree de ANTLR en el AST propio. Se emplean los métodos `exitX` y no los `enterX` porque el recorrido del Listener es en post orden: cuando se ejecuta `exitX`, todos los hijos de esa regla ya fueron procesados y sus nodos correspondientes ya existen. Esa propiedad es exactamente la que se necesita para construir el árbol de abajo hacia arriba.

Como los métodos del Listener no devuelven valor, los resultados intermedios deben almacenarse en una estructura externa. Esta implementación utiliza `ParseTreeProperty`, que asocia cada nodo del parse tree con el nodo del AST que le corresponde. Se elige esta estructura sobre una pila por una razón concreta: numerosas reglas de la gramática contienen elementos opcionales, como la dimensión de un arreglo o su lista de valores iniciales. Con una pila habría que determinar cuantos elementos extraer en cada caso según que partes estuvieron presentes; con el mapa basta consultar directamente por el hijo que interesa y recibir un valor nulo si no apareció.

6. Tabla de Símbolos y Análisis Semántico

6.1 Símbolo y Ambito

La clase `Simbolo` representa una entrada de la tabla y almacena el nombre, el tipo, el rol, el ámbito y la posición de la declaración. Los roles posibles se definen en el enum `RolSimbolo`: `VARIABLE`, `ARREGLO`, `PARAMETRO`, `FUNCION`, `ESTRUCTURA` y `CAMPO`.

La clase `Ambito` representa un nivel de alcance. Contiene un mapa de símbolos y una referencia al ámbito que lo contiene. La búsqueda de un nombre recorre esa cadena hacia arriba, de modo que una función puede ver las variables globales pero el ámbito global no ve las locales de ninguna función. Esa es precisamente la propiedad que permite que dos funciones distintas declaren variables con el mismo nombre sin que se produzca un conflicto.

Se emplea `LinkedHashMap` y no `HashMap` porque conserva el orden de inserción, que es el orden en que conviene mostrar la tabla al graficarla.

6.2 TablaSimbolos

La clase TablaSimbolos gestiona la pila de ambitos y expone las operaciones fundamentales:

Metodo	Comportamiento
entrarAmbito(nombre)	Crea un nivel nuevo cuyo padre es el ambito actual
salirAmbito()	Regresa al ambito padre, cerrando el actual
declarar(simbolo)	Agrega un simbolo al ambito actual. Devuelve falso si ya existe uno con ese nombre en el mismo nivel
buscar(nombre)	Busca subiendo por la cadena de ambitos hasta el global
buscarLocal(nombre)	Busca unicamente en el ambito actual
buscarFuncion(nombre)	Busca y verifica que el simbolo encontrado sea una funcion
buscarEstructura(nombre)	Busca y verifica que el simbolo encontrado sea una estructura
getHistorial()	Devuelve todos los simbolos declarados durante el analisis, en orden de aparicion

Ademas de la estructura jerarquica de ambitos, la clase mantiene una lista plana con todos los simbolos declarados a lo largo del analisis. Esa lista es la que alimenta el reporte grafico de la tabla de simbolos, porque conserva incluso los simbolos de ambitos que ya se cerraron y que por lo tanto ya no son alcanzables desde la cadena de busqueda.

6.3 Analizador semantico y estrategia de pasadas

La clase AnalizadorSemantico recorre el AST en cinco pasadas sucesivas. El orden no es arbitrario: cada pasada depende de la informacion registrada por las anteriores.

Pasada	Contenido	Justificacion del orden
1	Estructuras globales	Una variable o un parametro pueden declararse del tipo de una estructura, asi que las definiciones deben existir antes de validar cualquier declaracion.
2	Firmas de funciones	Una variable global puede inicializarse con el resultado de una llamada a funcion, y una funcion puede invocar a otra definida mas abajo en el archivo.
3	Variables globales	Ya se conocen las estructuras y las firmas, de modo que sus inicializadores se pueden validar por completo.
4	Cuerpos de funciones	Se abre un ambito por funcion, se registran los parametros y las variables locales, y se validan las instrucciones.
5	Seccion principal	Se validan las instrucciones de la seccion MAIOR con toda la informacion disponible.

Un mecanismo complementario evita la generacion de mensajes redundantes. Cuando una expresion resulta invalida, el analizador devuelve el tipo especial ERROR, que la clase TablaTipos trata como comodin compatible con todo. De ese modo un unico error real no desencadena una cascada de mensajes derivados que oscurecerian la causa original.

6.4 Sistema de tipos y jerarquia de conversion

El lenguaje define cinco tipos primitivos organizados en una jerarquia. El resultado de una operacion entre dos tipos distintos adopta siempre el nivel mas alto de los dos, y una asignacion solo es valida si el tipo de destino es de nivel igual o superior al del origen.

Tipo	Nivel	Descripcion
textum	5	Cadena de caracteres. Solo admite el operador + como concatenacion
decimalis	4	Numero de punto flotante
numerus	3	Numero entero
littera	2	Caracter unico
bool	1	Valor booleano, verum o falsus

La conversion implicita opera unicamente hacia arriba en la jerarquia. Un valor de tipo numerus puede almacenarse en una variable decimalis porque no hay perdida de informacion, pero la operacion inversa se rechaza porque los decimales se perderian de forma silenciosa.

6.5 Tabla de compatibilidad de tipos

La siguiente tabla resume el tipo resultante de aplicar cada categoria de operador. La celda indica el tipo del resultado, o la palabra error cuando la combinacion no es valida.

Operacion	Operandos	Resultado
+	textum con cualquier tipo	textum, por concatenacion
+, -, *, /	numericos entre si	El tipo de mayor nivel de los dos
-, *, /	textum con cualquier tipo	error
<, >, <=, >=	numerus, decimalis, littera	bool
<, >, <=, >=	textum o bool involucrados	error
==, !=	tipos iguales o numericos mezclados	bool

Operacion	Operandos	Resultado
&&,	bool con bool	bool
&&,	cualquier otra combinacion	error
non	bool	bool
non	cualquier otro tipo	error
- unario	numerus, decimalis	El mismo tipo del operando
++, --	numerus, decimalis	El mismo tipo del operando
Cualquiera	arreglo o estructura completa	error

La ultima fila recoge una regla general del lenguaje: los arreglos y las estructuras no se operan como unidades. Para usarlos en una expresion es necesario acceder primero a una posicion o a un atributo.

6.6 Validaciones implementadas

El analizador semantico aplica las siguientes verificaciones, todas exigidas por el enunciado de la practica:

Categoria	Verificaciones
Declaraciones	Variable o funcion no declarada. Nombre declarado dos veces en el mismo ambito. Declaracion fuera de la seccion VARIABLES o dentro de un bloque de control.
Tipos	Compatibilidad segun la tabla anterior. Conversion implicita valida. Concatenacion como unica operacion admitida sobre textum.
Estructuras	Nombres de atributo sin repetir en la definicion. Todos los atributos inicializados al instanciar. Atributos inexistentes. Tipos de atributo correctos. Orden libre de los atributos.
Arreglos	Cantidad de valores iniciales menor o igual a la dimension. Tipo de los valores. Dimension numerica y mayor que cero. Indice numerico, no negativo y dentro de rango cuando es evaluable durante el analisis.
Funciones	Cantidad y tipo de los argumentos. Tipo de retorno coincidente con el declarado. Todos los caminos retornan un valor en las funciones ratio.Codigo inalcanzable despues de un reddere. Parametros sin nombres repetidos.
Control de flujo	Condiciones estrictamente booleanas en si, dum, facere y per. Las instrucciones perge e interrompe solo dentro de un

Categoria	Verificaciones
	ciclo. La instruccion reddere solo dentro de una funcion.
Entrada y salida	El destino de una lectura debe ser asignable. No se puede leer ni imprimir un arreglo o una estructura como unidad.

La verificación de que todos los caminos retornan merece una nota. Un condicional solo cuenta como camino completo si posee una rama aliter final y todas sus ramas retornan; de lo contrario existe una ruta de ejecucion que abandona la funcion sin devolver valor.

7. Simulacion de la Pila de Analisis

El enunciado solicita mostrar la pila de llamadas del analizador en tiempo real, con navegacion paso a paso y un registro de las operaciones realizadas. Es necesario precisar el alcance de esta funcionalidad.

ANTLR 4 emplea analisis LL(*), que es descendente. Las operaciones de desplazamiento y reduccion son propias de los analizadores ascendentes de tipo LR, de modo que ANTLR no las ejecuta realmente. Lo que se implementa es una simulacion construida a partir de los eventos que emite el Listener durante el recorrido del parse tree, tal como confirmo el auxiliar del curso al responder esta duda en el foro.

La clase RegistradorPila implementa la interfaz ParseTreeListener y traduce los eventos del recorrido en operaciones de pila:

Evento	Operacion	Efecto sobre la pila
visitTerminal	SHIFT	Se apila el lexema del token leido de la entrada
enterEveryRule	Ninguna	Se anota la altura actual de la pila para saber donde empiezan los simbolos de esta regla
exitEveryRule	REPLACE	Se desapila todo lo acumulado desde esa altura y se apila en su lugar el nombre del no terminal
Cierre de la regla inicial	ACCEPT	Se registra que la entrada pertenece al lenguaje

Cada operacion genera un objeto PasoPila que almacena el numero de paso, la operacion, el simbolo involucrado, la linea del codigo fuente y una copia completa del contenido de la pila en ese instante. La copia es indispensable: si se guardara una referencia a la pila real, al retroceder en la navegacion se mostraria siempre el estado final del analisis en lugar del estado historico correspondiente.

La ventana VentanaPila presenta el historial completo en una tabla que funciona simultaneamente como registro de operaciones y como navegador. Seleccionar una fila muestra el estado de la pila en ese momento, y los botones desplazan la seleccion. Las filas se colorean segun la operacion: azul para SHIFT, rosa para REPLACE y verde para ACCEPT.

8. Traduccion a PigLatin

8.1 Leyes de traduccion

El enunciado exige transformar el codigo fuente al idioma PigLatin aplicando tres leyes. Se traducen los identificadores y las palabras reservadas; los simbolos del lenguaje, los literales numericos y el contenido de las cadenas de texto permanecen intactos.

Ley	Regla	Ejemplos
Consonantes	Si la palabra comienza con una o mas consonantes, se trasladan al final y se agrega el sufijo ay	fuerza produce uerzafay, tabla produce ablatay
Vocales	Si la palabra comienza con vocal, se agrega el sufijo way sin mover nada	inicio produce inicioway, archivo produce archivoway
Porcina	Las funciones especiales cambian de simbolo	El operador de lectura produce %OINK_OINK y el de impresion produce %OINK

La clase ReglasPigLatin implementa estas leyes. Cuando la palabra original esta escrita completamente en mayusculas, el resultado tambien se convierte a mayusculas, lo que mantiene legibles los marcadores de seccion del programa.

8.2 Implementacion por recorrido del arbol

El enunciado establece una restriccion explicita: la traduccion debe realizarse recorriendo el AST y no se acepta una implementacion basada en reemplazo de texto ni en expresiones regulares.

La clase TraductorPigLatin cumple esa restriccion regenerando el codigo desde el arbol. Recorre cada nodo, traduce los identificadores y las palabras reservadas que le corresponden, y reconstruye la sintaxis completa con su sangria. En ningun momento accede al texto original del archivo.

Un detalle de la reconstruccion de expresiones ilustra por que el recorrido del arbol es superior al reemplazo textual. La precedencia de los operadores esta representada por la forma del arbol, no por los parentesis del codigo original. Al regenerar, el traductor calcula la precedencia de cada operacion y agrega parentesis unicamente donde son necesarios para preservar el significado. Un reemplazo textual no podria hacer esa distincion porque carece de la informacion estructural.

9. Reportes del Sistema

9.1 Reporte de errores

La clase RecolectorErrores centraliza los errores de las tres fases y los ordena por línea y columna, sin importar en qué etapa se detectaron. Cada entrada incluye el tipo, la posición exacta, el lexema involucrado cuando aplica y una descripción en español.

La clase EscuchaErrores intercepta los mensajes que emite ANTLR y los traduce a formulaciones comprensibles para el usuario. Los mensajes originales de la librería están en inglés y emplean terminología interna del generador.

9.2 Reporte del Árbol de Sintaxis Abstracta

La clase GeneradorDot recorre el AST utilizando los métodos etiqueta y hijos definidos en la clase base, y produce el código DOT correspondiente. La clase VisorDot invoca a Graphviz para generar tres archivos en la carpeta reportes:

- El archivo con extensión dot, que contiene el código fuente del gráfico.
- El archivo con extensión png, que se muestra dentro de la aplicación.
- El archivo con extensión svg, en formato vectorial, útil cuando el árbol es muy extenso porque no pierde calidad al ampliarse.

Los nodos se colorean según su naturaleza: azul para los nodos internos y amarillo para las hojas, es decir, los literales, los identificadores y los parámetros. De ese modo se distingue de un vistazo donde termina cada rama del árbol.

Si Graphviz no está instalado en el sistema, el archivo dot se genera de todos modos y la aplicación ofrece copiarlo al portapapeles para pegarlo en un servicio de renderizado en línea. El reporte nunca queda inaccesible por una dependencia externa.

9.3 Reporte de la tabla de símbolos

La tabla de símbolos se grafica mediante etiquetas HTML de Graphviz, que permiten construir celdas alineadas sin calcular posiciones manualmente. Se muestran el nombre, el tipo, el rol, el ámbito, la línea y la columna de cada símbolo declarado durante el análisis.

9.4 Reporte de la pila de análisis

La ventana de la pila permite graficar el estado correspondiente a cualquier paso seleccionado. La cima se dibuja en la parte superior y se resalta con un color distinto, siguiendo la convención habitual de representación de pilas.

10. Interfaz Grafica

10.1 Componentes principales

Componente	Funcion
VentanaPrincipal	Ventana principal con editor en pestanas, consola de salida, barra de estado y barra de menu
ResaltadorSintaxis	Coloreado de sintaxis en tiempo real utilizando el lexer generado por ANTLR
GestorArchivos	Apertura, guardado y descarga de archivos con extension lat y pig
VisorDot	Ventana de reportes graficos con pestanas para la imagen, el vectorial y el codigo DOT
VentanaPila	Navegacion paso a paso del analisis con registro de operaciones coloreado

10.2 Coloreado de sintaxis

El coloreado no emplea expresiones regulares. La clase ResaltadorSintaxis reutiliza el mismo CodexLatinusLexer que ANTLR genera a partir de la gramatica: cada token que produce el lexer trae consigo su tipo y sus posiciones inicial y final, que es precisamente la informacion necesaria para pintar el texto.

La ventaja practica de este enfoque es que el coloreado nunca puede desincronizarse del lenguaje. Si en el futuro se agrega una palabra reservada a la gramatica, el editor la reconoce sin modificar una sola linea de la clase del resaltador.

La unica porcion que recorre el texto directamente es la deteccion de comentarios, y por una razon concreta: en la gramatica los comentarios llevan la instruccion skip, lo que significa que el lexer nunca los entrega en el flujo de tokens. La busqueda respeta que una secuencia de dos almohadillas dentro de una cadena de texto no abre un comentario.

La paleta de colores utilizada es la siguiente:

Categoria	Estilo
Marcadores de seccion	Morado, negrita
Palabras reservadas	Azul, negrita
Tipos de datos	Turquesa, negrita
Cadenas y caracteres	Verde
Numeros y booleanos	Naranja
Operadores	Rojo oscuro
Comentarios	Gris, cursiva

El editor emplea el componente JTextPane en lugar de JTextArea porque este ultimo no admite atributos de color por caracter. El coloreado se dispara con cada modificacion del documento, pero de forma diferida mediante invokeLater: modificar los atributos de un documento mientras este notifica un cambio provoca una excepcion de estado.

11. Herramientas y Configuracion del Proyecto

11.1 Tecnologias utilizadas

Tecnologia	Version	Uso en el proyecto
Java	21	Lenguaje de implementacion del compilador
ANTLR	4.13.2	Generacion del analizador lexico y sintactico
antlr4-maven-plugin	4.13.2	Integracion de la generacion en el ciclo de construccion
Apache Maven	3.x	Gestion de dependencias y construccion
Java Swing	Incluido en el JDK	Interfaz grafica de escritorio
Graphviz	Externo	Renderizado de los reportes graficos
Apache NetBeans	31	Entorno de desarrollo
Git	Externo	Control de versiones

11.2 Configuracion de Maven

El archivo pom.xml declara la dependencia del runtime de ANTLR y configura el plugin que genera el analizador a partir de la gramatica:

```
<dependency>
  <groupId>org.antlr</groupId>
  <artifactId>antlr4-runtime</artifactId>
  <version>4.13.2</version>
</dependency>

<plugin>
  <groupId>org.antlr</groupId>
  <artifactId>antlr4-maven-plugin</artifactId>
  <version>4.13.2</version>
  <configuration>
    <listener>true</listener>
    <visitor>true</visitor>
    <sourceDirectory>${basedir}/src/main/antlr4</sourceDirectory>
  </configuration>
  <executions>
    <execution>
      <phase>generate-sources</phase>
      <goals><goal>antlr4</goal></goals>
    </execution>
  </executions>
</plugin>
```



```
</executions>
</plugin>
```

Es necesario distinguir dos artefactos que comparten el nombre ANTLR. El plugin se ejecuta durante la compilación y genera las clases Java a partir del archivo de gramática. El runtime es la biblioteca que esas clases generadas necesitan para funcionar durante la ejecución, y de la que provienen tipos como Parser, Token y ParseTreeWalker.

La ubicación del archivo de gramática determina el paquete de las clases generadas. El archivo se encuentra en la ruta `src/main/antlr4/com/mycompany/codexlatinus/gramatica`, por lo que las clases resultantes pertenecen al paquete `com.mycompany.codexlatinus.gramatica`.

11.3 Construcción y ejecución

Para compilar el proyecto completo:

```
mvn clean compile
```

El uso de `clean` es obligatorio cada vez que se modifica el archivo de gramática. El plugin realiza una construcción incremental y omite el reprocesamiento si detecta que el archivo no ha cambiado, lo que puede llevar a trabajar contra un analizador desactualizado.

Para ejecutar la aplicación:

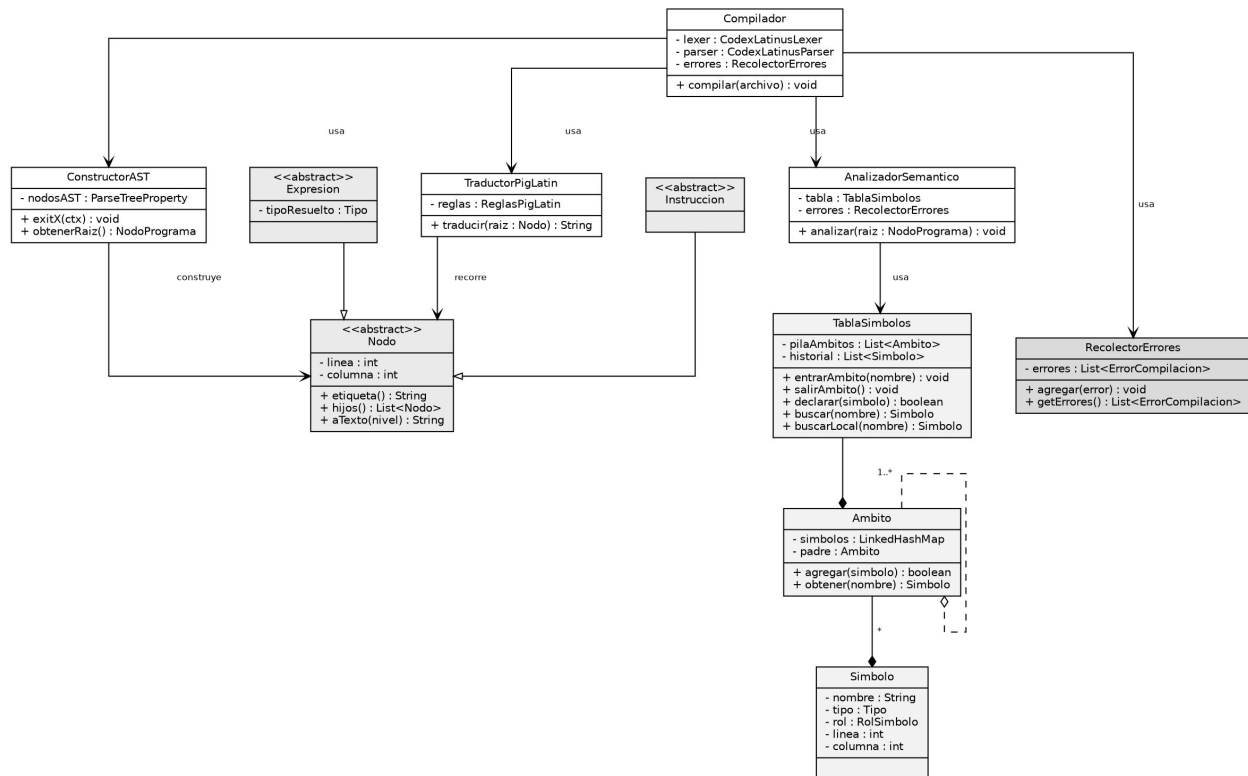
```
mvn exec:java -Dexec.mainClass=com.mycompany.\
practica_1_compi_2_mynor_monzon_1s_2026.\
Practica_1_Compi_2_Mynor_Monzon_1S_2026
```

11.4 Estructura de carpetas del repositorio

```
Practica_1_Compi_2_Mynor_Monzon_1S_2026/
  pom.xml
  src/main/antlr4/...      -> Gramatica CodexLatinus.g4
  src/main/java/...        -> Código fuente del compilador
  entradas/                -> Archivos de prueba .lat
    validos/               -> Deben compilar sin errores
    errores/               -> Deben reportar errores específicos
  reportes/                -> Salida generada: .dot, .png, .svg
  documentacion/           -> Manual técnico y manual de usuario
```

12. Diagrama de Clases

El siguiente diagrama muestra la relacion entre las clases principales del compilador, agrupadas por paquete.



13. Pruebas Realizadas

El compilador se verificó con un conjunto sistemático de archivos de prueba organizados en dos categorías. Los archivos de la carpeta válidos deben compilar sin ningún error; los de la carpeta errores deben reportar exactamente los errores documentados en cada uno.

Archivo	Resultado	Cobertura
basico.lat	0 errores	41 secciones que ejercitan cada concepto del lenguaje de forma aislada
avanzado.lat	0 errores	12 programas no triviales: ordenamiento burbuja, árbol binario de tres niveles, Ackermann con recursión doble, matriz simulada
errores.lat	74 errores	Errores semánticos de todas las categorías, cada uno documentado con su causa y reporte esperado

Archivo	Resultado	Cobertura
e01_lexicos.lat	4 lexicos	Caracteres que no pertenecen al alfabeto del lenguaje
e02_sintacticos.lat	3 sintacticos	Verificacion del modo panico: se reporta mas de un error en una sola pasada
e03 a e08	59 semanticos	Tipos, declaraciones, estructuras, arreglos, funciones y control de flujo

Durante la ejecucion de estas pruebas se detectaron y corrigieron tres defectos: los errores lexicos se clasificaban erroneamente como sintacticos, el orden de las pasadas del analizador semantico impedia inicializar una variable global con una llamada a funcion, y la gramatica no admitia fijar la dimension de un arreglo de tipo primitivo dentro de una instancia de estructura.